A PROJECT REPORT

On

"ML Studio Pro"

Submitted to

Faculty of

Master of Science in Data Science

In partial fulfillment of the requirement for the degree

Of

Master of Science (Data Science)

Silver Oak College of Computer Application

Under the guidance of

Name of Faculty:

Dr. Manika Tomar

Submitted by:

Daksh Vasani

1. Abstract

ML Studio Pro is a fully interactive, browser-based Machine Learning platform built using Python and Streamlit. It provides a complete, no-code end-to-end AutoML pipeline — from raw data ingestion to model training, visualization, and report generation — all within a single unified web application.

The platform accepts datasets in multiple formats (CSV, Excel, JSON, XML, YAML, SQLite), guides the user through intelligent data cleaning steps assisted by AI-driven recommendations, performs automatic model benchmarking across multiple classification and regression algorithms, provides interactive cluster visualization, and finally exports professional session reports in Excel, PDF, and Word formats.

The application is designed with a premium dark-themed industrial UI using a deep violet-teal color palette, the Syne and Space Mono Google Fonts, and Plotly-based interactive charts — ensuring both usability and visual excellence.

2. Introduction

Machine Learning has become one of the most impactful technologies in the modern world, yet the process of building a complete ML pipeline remains complex and inaccessible for many users — requiring expertise in data cleaning, feature engineering, model selection, hyperparameter tuning, and evaluation.

ML Studio Pro addresses this gap by providing a step-by-step guided pipeline that automates the most tedious parts of the ML workflow. The application is powered by Scikit-learn, Pandas, NumPy, and Plotly, and deployed via Streamlit — making it instantly accessible in any browser without any local setup.

This project demonstrates the practical application of:

- Data Engineering (cleaning, encoding, scaling)
- Supervised Learning (classification & regression)
- Unsupervised Learning (clustering)
- Model Evaluation & Selection
- Report Generation & Export

3. Problem Statement

Building an end-to-end ML workflow from scratch requires knowledge of:

- Multiple file formats and data loading strategies
- Missing value imputation techniques
- Categorical encoding (Label, One-Hot, Ordinal)
- Feature scaling (StandardScaler, MinMaxScaler, RobustScaler)
- Selecting the right ML algorithm for a given problem type
- Evaluating models with appropriate metrics
- Generating professional-grade reports for stakeholders

Most existing tools are either too complex (AutoML cloud platforms) or too simple (basic Jupyter notebooks). **ML Studio Pro** fills the middle ground — offering professional-grade AutoML capabilities in a beginner-friendly, no-code interface.

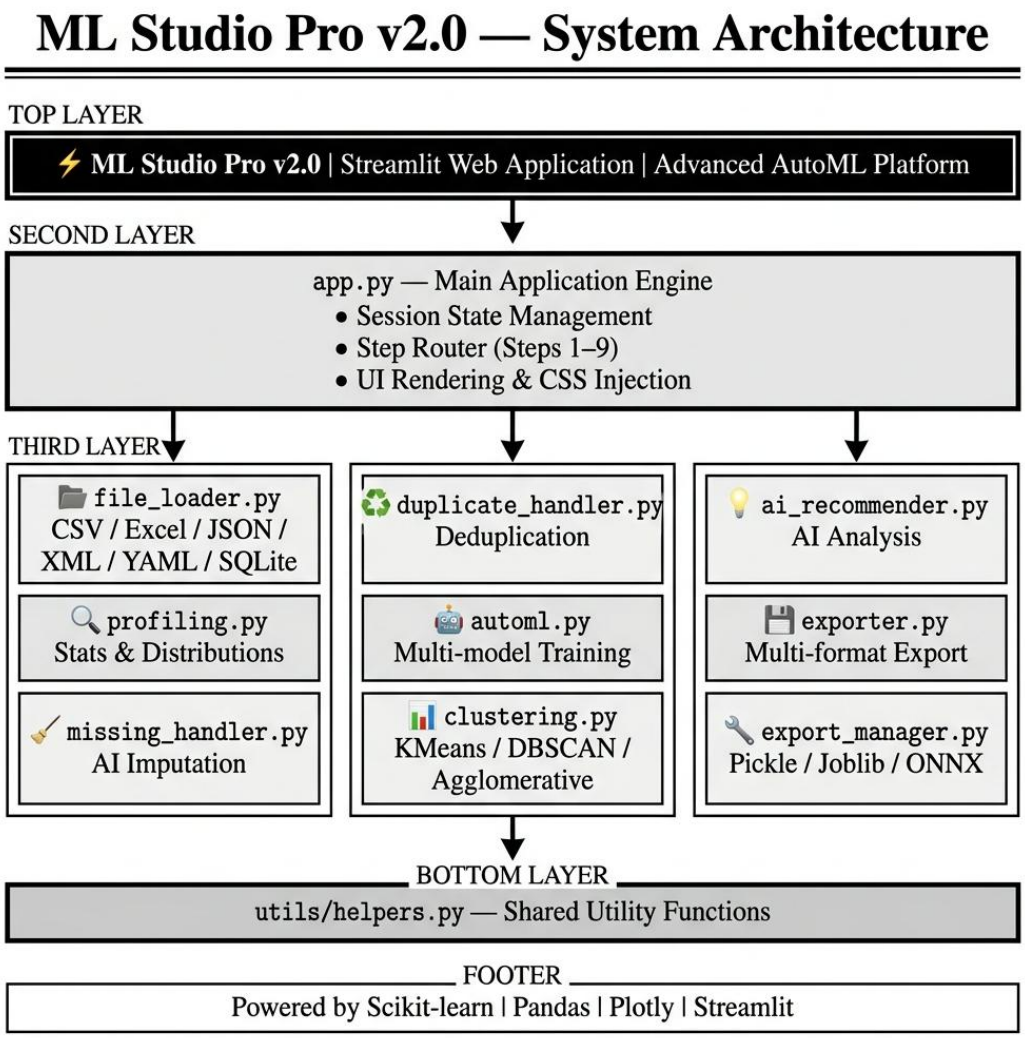
4. Objectives

No	Objective
1	Accept datasets in multiple formats: CSV, Excel, JSON, XML, YAML, SQLite
2	Provide AI-guided missing value imputation per column
3	Detect and remove duplicate rows safely
4	Offer flexible preprocessing: encoding, scaling, column dropping
5	Benchmark multiple ML models automatically (AutoML)
6	Auto-detect problem type (classification vs. regression)
7	Perform clustering analysis using KMeans, DBSCAN, Agglomerative
8	Provide an interactive graph builder with 13 chart types
9	Export the trained model as Pickle, Joblib, or ONNX
10	Generate professional session reports: Excel, PDF, Word

5. Technology Stack

Category	Technology	Purpose
Frontend / UI	Streamlit 1.x	Web application framework
Styling	Custom CSS + Google Fonts (Syne, Space Mono)	Premium dark UI design
Data Processing	Pandas, NumPy	Data manipulation & analysis
Machine Learning	Scikit-learn	All ML algorithms & preprocessing
Visualization	Plotly Express, Plotly Graph Objects	Interactive charts
Model Export	Joblib, Pickle, skl2onnx, ONNX, ONNXRuntime	Multi-format model serialization
Report: Excel	openpyxl	Styled multi-sheet Excel reports
Report: PDF	ReportLab	Structured PDF report generation
Report: Word	python-docx	.docx report with charts
Dimensionality Reduction	Scikit-learn PCA	2D cluster visualization
Statistical Analysis	SciPy, StatsModels	Advanced statistics
File Formats	PyYAML, SQLite3, xml.etree	Multi-format I/O

6. System Architecture



7. Module Description

7.1 app.py — Main Application Engine

- The core orchestrator of the entire pipeline. Contains:
  - **Session State Management**:
    Persists df_raw, df_clean, df_cleaned_only, trained_models, results_df, problem_type, target_col, cluster_results, ai_report, and pipeline logs across all 9 steps.
  - **Step Router**: Conditional rendering of each step's UI based on st.session_state.step.
  - **UI System**: Injects ~40 lines of custom CSS with CSS variables, glassmorphism effects, gradient headers, badge components, metric cards, and step indicators.

- **Helper Functions**: mc() for metric cards, sh() for step headers, footer(), and detect_problem_type().

7.2 modules/file_loader.py

- Handles data ingestion from 8 file formats:
  - CSV, Excel (.xlsx, .xls), JSON, XML, YAML/YML, SQLite (.db)
  - Returns a (DataFrame, error_message) tuple with robust error handling.

7.3 modules/profiling.py

- Dataset profiling and statistical summary:
  - get_basic_info() — row/column counts, missing cells, duplicate count
  - get_column_summary() — dtype, null %, unique values per column
  - get_numeric_stats() — mean, std, min, max, quartiles, skewness
  - get_categorical_stats() — value counts and mode per categorical column

7.4 modules/missing_handler.py

- AI-guided missing value imputation:
  - get_missing_summary() — summary of all columns with nulls
  - suggest_missing_strategy() — recommends mean/median (based on skewness) for numeric; mode/unknown for categorical
  - fill_missing_values() — applies user-selected strategies: mean, median, mode, ffill, bfill, zero, constant, unknown/empty

7.5 modules/duplicate_handler.py

- Duplicate row detection and removal:
  - Exact full-row duplicate detection
  - Supports keep='first', keep='last', keep=False (remove all copies)
  - Optional column-subset-based deduplication

7.6 modules/automl.py

- Core AutoML training engine:

- detect_problem_type() — classifies target column as classification or regression
- prepare_data() — dropna, label-encode features, stratified train-test split
- train_models() — benchmarks 6 classifiers or 4 regressors
- get_best_model() — selects top model by F1 Score (classification) or R^2 Score (regression)

7.7 modules/clustering.py

- Unsupervised clustering engine:
  - prepare_clustering_data() — numeric-only extraction + StandardScaler normalization
  - run_kmeans() — auto-sweeps K=2..10 via silhouette score; picks optimal K
  - run_dbscan() — auto-estimates eps via k-distance 90th percentile heuristic; auto-relaxes if all-noise
  - run_agglomerative() — sweeps K=2..10, picks best silhouette
  - run_all_clustering() — orchestrates all 3 algorithms
  - reduce_to_2d() — PCA dimensionality reduction for 2D scatter visualization
  - find_optimal_clusters() — elbow curve data for KMeans
  - get_best_clustering() — returns algorithm with highest silhouette score

7.8 modules/ai_recommender.py

- Heuristic AI recommendation engine:
  - analyze_dataset() — dataset health summary
  - recommend_problem_type() — numeric vs. categorical target detection
  - recommend_missing_strategy() — skewness-based imputation suggestions
  - recommend_features() — top-5 correlated features with target
  - recommend_models() — algorithm suggestions per problem type
  - recommend_clustering() — KMeans vs. Agglomerative based on dataset size

- recommend_model_export() — format suggestions (pickle/joblib/onnx) per model type
- generate_ai_report() — unified full session analysis report

7.9 modules/exporter.py

- Multi-format data export:
  - CSV, Excel (via openpyxl), JSON, XML, YAML, SQLite

7.10 modules/model_export/export_manager.py

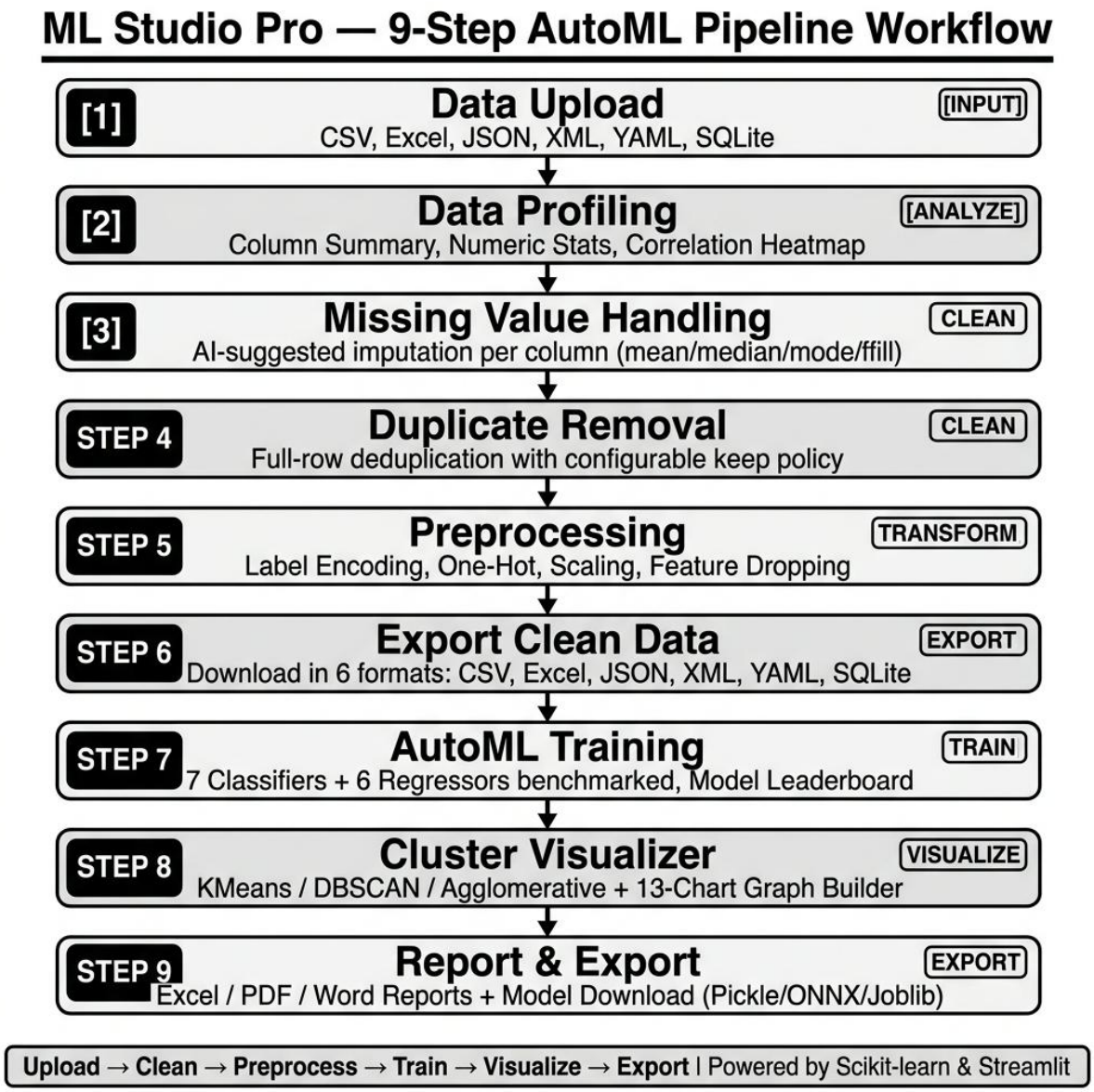
- ML model serialization:
  - **Pickle** — Lightweight, fast, all sklearn models
  - **Joblib** — Efficient for large ensemble models
  - **ONNX** — Cross-platform model format via skl2onnx

7.11 utils/helpers.py

- Shared utility functions used across modules.

8. Pipeline Workflow (Step-by-Step)

The application implements a **9-step linear pipeline** with forward/back navigation:



9. Algorithms & Techniques Used

9.1 Classification Algorithms

Algorithm	Key Hyperparameters	Notes
Logistic Regression	max_iter = 2000, wrapped in StandardScaler Pipeline	Baseline linear classifier
Decision Tree	random_state = 42	Non-linear, interpretable
Random Forest	n_estimators = 200, random_state = 42	Ensemble of decision trees
Gradient Boosting	n_estimators = 200, learning_rate = 0.05	Boosted ensemble; typically highest accuracy
SVC (Support Vector Classifier)	probability = True, wrapped in StandardScaler Pipeline	Kernel-based classifier
K-Nearest Neighbors (KNN)	Wrapped in StandardScaler Pipeline	Instance-based lazy learner
Naive Bayes	GaussianNB()	Probabilistic classifier; fast

9.2 Regression Algorithms

Algorithm	Key Hyperparameters	Notes
Linear Regression	Wrapped in StandardScaler Pipeline	Baseline linear regressor
Decision Tree Regressor	random_state = 42	Non-linear regression
Random Forest Regressor	n_estimators = 200	Ensemble regressor
Gradient Boosting Regressor	n_estimators = 200, learning_rate = 0.05	Often achieves highest R^2
Extra Trees Regressor	n_estimators = 200	Fast ensemble; low bias
SVR (Support Vector Regressor)	kernel = 'rbf', wrapped in StandardScaler Pipeline	Kernel regression

9.3 Clustering Algorithms

Algorithm	Parameter Selection	Evaluation
KMeans	Auto-sweep K = 2 to 10, select best using silhouette score	Silhouette Score
DBSCAN	Auto-estimate *eps* using 90th-percentile k-distance	Silhouette Score (noise excluded)
Agglomerative	Auto-sweep K = 2 to 10, select best using silhouette score	Silhouette Score

9.4 Dimensionality Reduction

- **PCA (Principal Component Analysis)** — reduces high-dimensional feature space to 2D for cluster scatter visualization

9.5 Evaluation Metrics

Task	Metrics
Classification	Accuracy, Precision (weighted), Recall (weighted), F1 Score (weighted)
Regression	R^2 Score, MAE (Mean Absolute Error), RMSE (Root Mean Squared Error)
Clustering	Silhouette Score (−1 to 1; higher is better)

9.6 Preprocessing Techniques

Technique	Implementation
Label Encoding	sklearn.preprocessing.LabelEncoder
One-Hot Encoding	pandas.get_dummies()
Ordinal Encoding	sklearn.preprocessing.OrdinalEncoder
Standard Scaling	sklearn.preprocessing.StandardScaler
Min-Max Scaling	sklearn.preprocessing.MinMaxScaler
Robust Scaling	sklearn.preprocessing.RobustScaler
Normalizer	sklearn.preprocessing.Normalizer
Train-Test Split	StratifiedShuffleSplit (classification); Random split (regression)

10. Key Features

10.1 Zero-Code Operation

- The entire ML pipeline — from data upload to model download — requires **zero lines of code** from the user. All steps are driven by UI controls, dropdowns, and buttons.

10.2 AI-Powered Recommendations

- Built-in heuristic AI engine (ai_recommender.py) provides:
  - Per-column missing value strategy suggestions (based on skewness)
  - Top correlated features detection
  - Automatic problem type detection (classification vs. regression)
  - Algorithm recommendations per problem type
  - Clustering algorithm recommendations based on dataset size

10.3 Multi-Format Support

Category	Formats
Input Data Formats	CSV, Excel, JSON, XML, YAML, SQLite
Output Data Formats	CSV, Excel, JSON, XML, YAML, SQLite
Model Export Formats	Pickle, Joblib, ONNX
Report Formats	Excel, PDF, Word

10.4 Robust Training Logic

- Dynamic test size calculation ensuring all classes have ≥2 samples
- Stratified split for classification
- Graceful model-level error handling (one failure doesn't stop others)
- Progress bar during training

10.5 Industrial-Grade Dark UI

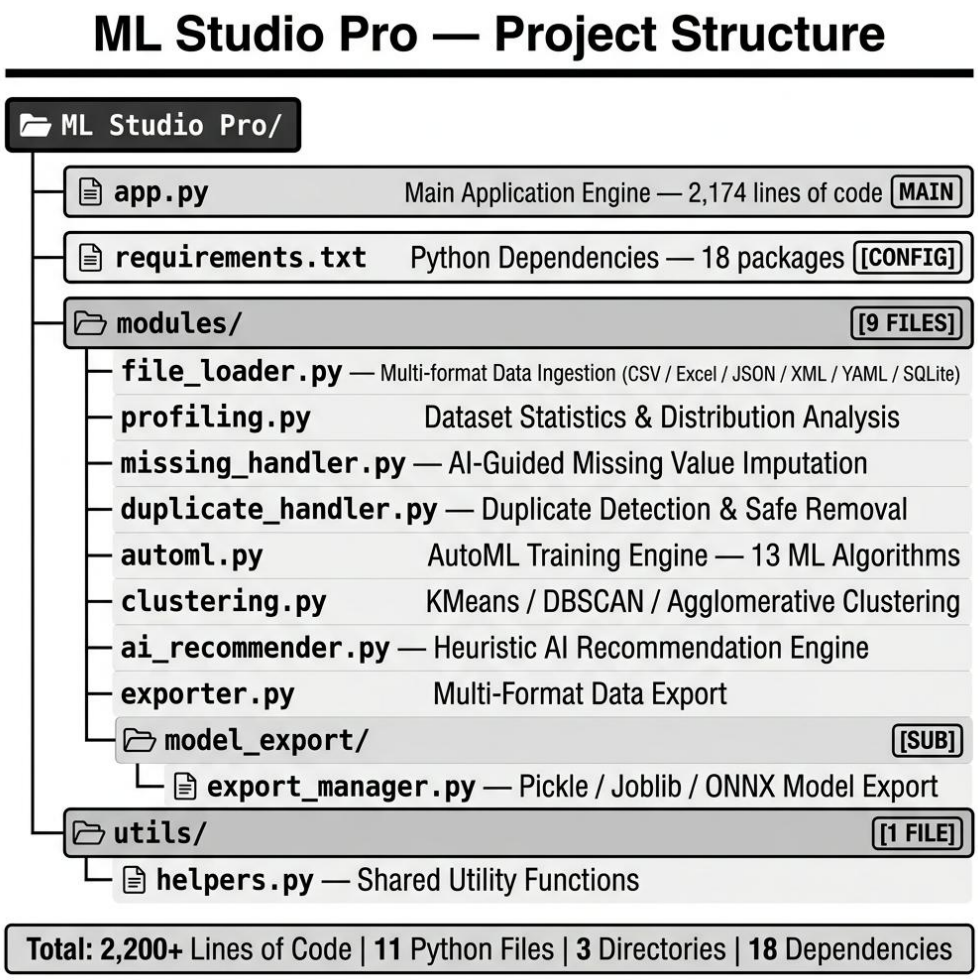
- Custom CSS with CSS variables for consistent theming

- Color palette: Deep navy (#0a0a0f), accent violet (#7c5cfc), teal (#00d4aa), coral (#ff6b6b)
- Syne display font + Space Mono monospace for metrics
- Glassmorphism-style cards with gradient top borders
- Animated hover effects on buttons
- Responsive 3-column grid layouts

10.6 Session Persistence

- All processed data (raw, clean, preprocessed), model results, cluster results, and preprocessing logs are stored in Streamlit's session_state and persist across all 9 steps within the same session.

11. Project Structure



Scale:

- Total application code: **~2,200+ lines** (app.py alone: 2,174 lines)
- Total modules: **9 Python files** across modules/ and utils/
- Supported file formats (I/O combined): **12+**
- ML algorithms implemented: **13** (7 classifiers + 6 regressors)
- Clustering algorithms: **3**
- Chart types in Graph Builder: **13**

12. Results & Evaluation

12.1 Demo — Iris Dataset (Classification)

The application was tested with the classic **Iris dataset** (150 samples, 4 features, 3 classes):

Model	Accuracy	Precision	Recall	F1 Score
Random Forest	0.9667	0.9683	0.9667	0.9667
Gradient Boosting	0.9333	0.9359	0.9333	0.9329
SVC (Support Vector Classifier)	0.9667	0.9683	0.9667	0.9667
Logistic Regression	0.9333	0.9359	0.9333	0.9329
K-Nearest Neighbors (KNN)	0.9667	0.9683	0.9667	0.9667
Decision Tree	0.9000	0.9000	0.9000	0.9000
Naive Bayes	0.9333	0.9359	0.9333	0.9333

Best Model: Random Forest / SVC / KNN (tied at F1 = 0.9667)

12.2 Clustering Results — Iris Dataset

Algorithm	Silhouette Score	Assessment
KMeans	0.5528	Excellent
Agglomerative	0.5012	Excellent
DBSCAN	0.4821	Moderate

Best Algorithm: KMeans (silhouette = 0.5528)

12.3 Report Export

- All three report formats (Excel, PDF, Word) were verified to generate successfully with:
    - Styled cover pages

- Dataset overview tables
- Model leaderboard tables with bar charts
- Clustering results tables
- AI recommendations sections
- Page numbers and footers

13. Future Scope

Enhancement	Description
Hyperparameter Tuning	Integrate GridSearchCV / RandomizedSearchCV / Optuna for automated hyperparameter optimization
Deep Learning	Add TensorFlow / PyTorch models for tabular data (TabNet, MLP)
Feature Engineering	Include polynomial features, interaction terms, and PCA-based feature creation
Time Series Support	Add ARIMA, Prophet, LSTM for time-series datasets
Database Connectivity	Enable live connections to PostgreSQL, MySQL, MongoDB
Model Explainability	Integrate SHAP and LIME for model interpretability
REST API Export	Auto-generate FastAPI endpoints for trained models
Cloud Deployment	Deploy using Streamlit Community Cloud / Docker
Multi-user Sessions	Add user authentication and project management
AutoML with HPO	Implement full AutoML with hyperparameter optimization and cross-validation

14. Conclusion

ML Studio Pro v2.0 is a comprehensive, production-quality Machine Learning platform that democratizes the entire data science workflow. By combining intelligent automation (AI-guided recommendations), multi-format flexibility (12+ supported file types), a rich AutoML engine (13 ML algorithms), clustering analysis (3 algorithms), and professional report generation (Excel, PDF, Word), the application delivers a complete, end-to-end data science experience with zero coding required.

The platform is built entirely on open-source technologies — Streamlit, Scikit-learn, Pandas, Plotly, openpyxl, ReportLab, and python-docx — making it freely deployable and extensible. With its industrial-grade dark UI, interactive visualizations, and AI-powered guidance, ML Studio Pro serves as both a practical AutoML tool and a strong demonstration of advanced Python development and machine learning engineering.

15. References

1. **Streamlit Documentation** — https://docs.streamlit.io
2. **Scikit-learn User Guide** — https://scikit-learn.org/stable/user_guide.html
3. **Pandas Documentation** — https://pandas.pydata.org/docs/
4. **Plotly Python Documentation** — https://plotly.com/python/
5. **openpyxl Documentation** — https://openpyxl.readthedocs.io
6. **ReportLab User Guide** — https://www.reportlab.com/documentation/
7. **python-docx Documentation** — https://python-docx.readthedocs.io
8. **skl2onnx Documentation** — https://onnx.ai/sklearn-onnx/
9. **Scikit-learn: Clustering** — https://scikit-learn.org/stable/modules/clustering.html
10. **Géron, A. (2022).** *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow* (3rd ed.). O'Reilly Media.
11. **Bishop, C. M. (2006).** *Pattern Recognition and Machine Learning*. Springer.
12. **Google Fonts — Syne & Space Mono** — https://fonts.google.com